# Multi-Agent Explainable AI Text Classification: A Comparative Study of Classical Machine Learning and Transformer-Based Models with SHAP and LIME

[Student Name]
*Department of Computer Engineering*
*TOBB University of Economics and Technology*
Ankara, Turkey
Student ID: [Student ID]
student@etu.edu.tr

***Abstract*—This paper investigates the following research question: can a multi-agent architecture effectively route and classify multilingual text, and how do traditional machine learning classifiers compare to transformer-based deep learning models in terms of accuracy, F1-score, and interpretability when explained through SHAP and LIME? To answer this question, I designed a three-agent system for text classification where one agent routes the input, one classifies it, and one explains the prediction. I trained and tested eight models, including classical data mining classifiers (Naive Bayes, SVM, Random Forest, KNN, Logistic Regression, Decision Tree, XGBoost) and a deep learning model (DistilBERT/BERTurk), on four datasets: IMDB, AG News, Turkish Sentiment, and Turkish News. The results show that transformer models generally get the best accuracy, but simpler classical models like SVM and Logistic Regression are also competitive and much faster to train. SHAP and LIME explanations reveal that classical models are easier to interpret, while transformer explanations are less consistent. Class imbalance in Turkish Sentiment and the complex structure of Turkish words make both types of models harder to use on Turkish data.**

***Index Terms*—text classification, explainable AI, multi-agent systems, SHAP, LIME, BERT, data mining, Turkish NLP**

## I. Introduction

The amount of text data produced every day on the internet is enormous. Social media posts, news articles, product reviews, and emails are just a few examples. Being able to automatically classify this text is very useful and is one of the core problems in data mining. Text classification involves feature extraction, model selection, and evaluation on real-world datasets. The challenge today is not just getting good accuracy, but also understanding *why* a model makes a certain prediction.

In recent years, deep learning models like BERT [2] and DistilBERT [3] have achieved very high accuracy on text classification tasks. However, these models are hard to understand. At the same time, classical data mining classifiers like SVM and Naive Bayes are much more interpretable but are often assumed to be less accurate. This raises an important question: is the accuracy gain from transformers actually worth the loss in interpretability, and how big is the gap in practice?

To answer this, I also need explanation tools. Two popular methods are SHAP [4] and LIME [5]. SHAP assigns a score to each word showing how much it contributed to the prediction. LIME works by slightly changing the input text and observing how the prediction changes. Both methods work with any classifier, so I can use them to compare models fairly in terms of interpretability.

This brings me to the central research question of this project:

> *"Can a multi-agent architecture effectively route and classify multilingual text, and how do traditional machine learning classifiers compare to transformer-based deep learning models in terms of accuracy, F1-score, and interpretability when explained through SHAP and LIME?"*

To answer this question, I built a three-agent system. The multi-agent design is not the goal itself — it is the method I use to systematically compare eight different models across four datasets in a modular and reproducible way. One agent routes the input to the right model, one runs the classification, and one generates the explanation. I also included two Turkish datasets to test whether the findings generalize beyond English, since most similar studies only look at English.

The main contributions of this project toward answering the research question are:

- A systematic comparison of seven classical data mining classifiers and one transformer model across four datasets in two languages.
- SHAP and LIME explanations for all models, making it possible to compare not just accuracy but also interpretability across model families.
- An analysis of how class imbalance and Turkish morphology affect the accuracy and explainability of both classical and transformer models.

- A three-agent pipeline as the experimental framework that makes the comparison modular and extensible.

The rest of this paper is organized as follows. Section II reviews related work. Section III describes the datasets. Section IV explains the preprocessing steps, the multi-agent architecture, and the models. Section V shows the results. Section VI discusses the findings in relation to the research question. Section VII concludes the paper.

## II. Related Work

### A. Classical Machine Learning for Text Classification

Before deep learning became popular, text classification was mostly done with classical data mining and machine learning methods. Han, Kamber, and Pei [1] describe many of these techniques in their data mining textbook, including Naive Bayes, Decision Trees, and SVMs. Naive Bayes is simple and fast but assumes words are independent. SVMs work well for text because text features (like TF-IDF vectors) are high-dimensional and sparse, which is actually a good situation for SVMs. Logistic Regression and Random Forest are also commonly used. These classical data mining methods are still popular today because they are fast to train, easy to interpret, and work well even without a GPU.

### B. Deep Learning and Pre-trained Language Models

BERT [2] was introduced by Devlin et al. and changed how NLP is done. It is a large model pre-trained on huge amounts of text, and it can be fine-tuned for many tasks including text classification. The main advantage of BERT is that it understands context, meaning it knows that the same word can mean different things in different sentences. Sanh et al. [3] later released DistilBERT, which is a smaller and faster version of BERT that keeps most of its performance. This is useful when computational resources are limited. Minaee et al. [10] wrote a survey comparing many deep learning methods for text classification. They found that transformers generally outperform older models, but classical methods are still competitive when the dataset is small. Chen and Guestrin [6] introduced XGBoost, a powerful gradient boosting algorithm. While it is more commonly used for structured data, it can also be applied to text features like TF-IDF vectors.

### C. Explainable AI for NLP

Lundberg and Lee [4] proposed SHAP, which uses ideas from game theory to assign each feature a score that represents its contribution to the prediction. SHAP has some nice theoretical properties, such as consistency and local accuracy. For tree models, an efficient version called TreeSHAP exists. Ribeiro, Singh, and Guestrin [5] introduced LIME, which works differently. It creates many slightly modified versions of the input, runs the model on each, and fits a simpler linear model to understand the local behavior. Both SHAP and LIME are model-agnostic, meaning they can be used with any classifier. They have been used in NLP for tasks like sentiment analysis and fake news detection.

### D. Turkish NLP

Turkish NLP is less studied than English NLP. One reason is that Turkish has a very complex word structure. A single root word can have many different suffixes attached, which means the vocabulary is much larger compared to English. This makes it harder for classical models that rely on matching exact word forms. Schweter [7] released BERTurk, a BERT model that was pre-trained specifically on Turkish text. BERTurk gave much better results on Turkish tasks compared to using the English BERT model. In my project, I use BERTurk for the Turkish datasets to see if it also helps for multi-class news classification.

### E. Multi-agent Systems

The idea of using multiple agents to solve a complex task was described by Wooldridge and Jennings [12]. The basic idea is to break a problem into smaller parts, where each part is handled by a separate agent. This makes the system more modular and easier to maintain. In my project, I apply this idea to text classification by having one agent for routing, one for classification, and one for generating explanations. This way, each agent has a clear responsibility and can be updated independently.

## III. Dataset Description

### A. Data Sources

In this project, I used four publicly available datasets. Two of them are in English and two are in Turkish. The IMDB dataset [8] was created by Maas et al. and contains 50,000 movie reviews labeled as positive or negative. It is a widely used benchmark for binary sentiment classification. The AG News dataset [9] was used by Zhang et al. to evaluate character-level neural networks. It contains news article titles and descriptions from four categories: World, Business, Sports, and Sci/Tech. The Turkish Sentiment dataset is available on HuggingFace and contains product and social media reviews with three labels: positive, neutral, and negative. Finally, the Turkish News dataset contains full news articles from Turkish news websites, labeled with seven different topic categories.

### B. Data Characteristics

Table I gives a summary of all four datasets. They are quite different from each other in terms of size, number of classes, and average text length, which makes comparing models across them an interesting challenge.

TABLE I
Dataset Summary

| Property | IMDB | AG News | Tr. Sent. | Tr. News |
|---|---|---|---|---|
| Language | English | English | Turkish | Turkish |
| Domain | Movie | News | Product | News |
| Train samples | 25,000 | 120,000 | 440,679 | 3,920 |
| Test samples | 25,000 | 7,600 | 48,965 | 980 |
| # Classes | 2 | 4 | 3 | 7 |
| Avg. text len. (chars) | 1,325 | 236 | 140 | 1,981 |
| Balance | Balanced | Balanced | Imbalanced | Balanced |
| Source | [8] | [9] | HuggingFace | GitHub |

The class labels for each dataset are: IMDB — *positive*, *negative*; AG News — *World*, *Business*, *Sports*, *Sci/Tech*; Turkish Sentiment — *pozitif*, *notr*, *negatif*; Turkish News — *siyaset*, *saglik*, *teknoloji*, *kultur*, *spor*, *ekonomi*, *dunya*.

Table II shows the class distribution of the Turkish Sentiment dataset. This dataset is quite imbalanced: the negative class has only 11.5% of the training samples. This can cause classifiers to mostly ignore the negative class because it is so rare, which is why I use macro F1 as the main metric for this dataset.

TABLE II
CLASS DISTRIBUTION — TURKISH SENTIMENT (TRAINING SET)

| Class | Count | % |
|---|---|---|
| pozitif | 235,949 | 53.5% |
| notr | 153,825 | 34.9% |
| negatif | 50,905 | 11.5% |
| **Total** | **440,679** | **100%** |

### C. Ethical Considerations

All four datasets are publicly available and released by their original authors for research use. They do not contain any personally identifiable information. IMDB reviews are anonymized, AG News only contains published news text, and the Turkish Sentiment reviews were anonymized before release. The Turkish News dataset only contains article text and category labels. I did not collect any new data for this project.

## IV. METHODOLOGY

### A. Data Preprocessing

**Text cleaning.** The first step was to clean the raw text. IMDB reviews contain HTML tags like `<br/>` which I removed using a regular expression. I also converted all text to lowercase for the classical models, since words like "Good" and "good" should be treated the same way.

**TF-IDF vectorization for classical models.** For the classical machine learning models, I converted the cleaned text into TF-IDF feature vectors. The parameters I used for each dataset are shown in Table III. I used `sublinear_tf=True` which applies a log transformation to the term frequencies. This helps reduce the effect of very common words. I also included bigrams for most datasets, because pairs of words like "not good" carry more information than single words. For Turkish News, I used trigrams as well, because Turkish words often carry meaning through longer combinations. I did not apply stemming or lemmatization because TF-IDF already handles word frequency differences statistically, and the transformer models use subword tokenization anyway.

**Transformer tokenization.** For the IMDB and AG News datasets, I used the `distilbert-base-uncased` tokenizer with a maximum length of 256 tokens. For the Turkish datasets, I used the `dbmdz/bert-base-turkish-cased` (BERTurk) tokenizer with a maximum length of 128 tokens. I chose 128 for Turkish because the average text length is shorter there, and it also helps with memory usage during training.

TABLE III
TF-IDF VECTORIZER CONFIGURATION

| Parameter | IMDB | AG News | Tr. Sent. | Tr. News |
|---|---|---|---|---|
| max_features | 50,000 | 30,000 | 30,000 | 20,000 |
| ngram_range | (1,2) | (1,2) | (1,2) | (1,3) |
| min_df | 2 | 2 | 3 | 1 |
| max_df | 0.95 | 0.95 | 0.95 | 0.90 |
| sublinear_tf | True | True | True | True |

### B. Multi-Agent Architecture

My system has three agents that work together, as shown in Fig. 1.

**Agent 1 — Intent Classifier Agent.** This agent takes the raw user input and tries to figure out which dataset it belongs to. For example, if someone types a movie review, the agent should route it to the IMDB model. It does this using a small classifier trained on example queries. The output is a dataset label and a recommended model name.

**Agent 2 — Classification Agent.** This agent takes the dataset label and model name from Agent 1, loads the correct model, and runs the classification. It returns the predicted label and confidence scores for all classes.

**Agent 3 — XAI Agent.** This agent takes the original text and the prediction from Agent 2 and generates an explanation. For tree-based models (Random Forest, Decision Tree, XGBoost), I use TreeSHAP which computes exact Shapley values efficiently. For linear models (SVM, Logistic Regression, Naive Bayes), I use the linear SHAP explainer. For the transformer, I use KernelSHAP with 100 background samples since the model is a black box. In addition to SHAP, I also run LIME separately to get a second perspective on the important words.

Fig. 1. Multi-agent pipeline. Agent 1 routes incoming text; Agent 2 classifies; Agent 3 explains. [FIGURE PLACEHOLDER — replace with architecture diagram after training.]

### C. Model Configurations

I trained eight different classifiers on each dataset. Seven of them are classical data mining models and one is a transformer-based deep learning model. The hyperparameters I used are shown in Table IV. I chose these values based on the recommendations in the literature and some preliminary experiments.

TABLE IV
MODEL HYPERPARAMETER SUMMARY

| Model | Key Parameters | IMDB | AG News | Tr. Sentiment | Tr. News |
|---|---|---|---|---|---|
| Naive Bayes | alpha | 0.1 | 0.1 | 0.5 | 1.0 |
| SVM (LinearSVC) | C | 1.0 | 0.5 | 0.5 | 0.1 |
| Random Forest | n_estimators / max_depth | 200 / – | 200 / – | 200 / – | 200 / 15 |
| KNN | n_neighbors | 7 | 5 | 5 | 3 |
| Logistic Regression | C / solver | 1.0 / lbfgs | 1.0 / saga | 0.5 / saga | 0.1 / lbfgs |
| Decision Tree | max_depth / criterion | 20 / gini | 25 / gini | 20 / gini | 10 / entropy |
| XGBoost | n_estimators / depth / lr | 200 / 6 / 0.1 | 200 / 6 / 0.1 | 200 / 5 / 0.05 | 100 / 4 / 0.05 |
| Transformer | model† / epochs / lr | dBERT / 3 / 2e-5 | dBERT / 3 / 2e-5 | BTurk / 3 / 2e-5 | BTurk / 5 / 3e-5 |

† dBERT = `distilbert-base-uncased`; BTurk = `dbmdz/bert-base-turkish-cased`

### D. Experimental Setup

For classical models, I used `scikit-learn` (v1.4) and `XGBoost` (v2.0). I wrapped the TF-IDF vectorizer and each classifier into a single `Pipeline` object to make the code cleaner. For transformer models, I used the HuggingFace `transformers` library (v4.38) with `PyTorch` (v2.2). I fine-tuned the transformers using `AdamW` with a linear learning rate schedule and 10% of steps for warmup.

I used the original train/test splits from each dataset so that my results can be compared to other papers. I did not create a separate validation set. The random seed was set to 42 in all experiments so that the results are reproducible.

Hardware: [fill in: CPU/GPU specification]. Libraries: Python 3.10, `scikit-learn` 1.4, `XGBoost` 2.0, `transformers` 4.38, `PyTorch` 2.2, `shap` 0.45, `lime` 0.2.0.

### E. Explainability Methods

**SHAP.** SHAP computes a score for each word that shows how much it contributed to the prediction. For tree-based models, I use TreeSHAP [4] which is fast and gives exact values. For linear models like SVM and Logistic Regression, I use the linear SHAP explainer. For the transformer model, which is a black box, I use KernelSHAP with 100 background samples. I visualize the top-10 most important features as a bar chart.

**LIME.** LIME works by creating 5,000 slightly different versions of the input text (by randomly removing words), running the model on each version, and then fitting a simple linear model to see which words matter most. I use the `LimeTextExplainer` class and extract the top-10 positive and negative contributing words. I apply LIME to both the TF-IDF models and the transformer.

## V. RESULTS

### A. Evaluation Metrics

I evaluated all models using Accuracy, Precision (macro), Recall (macro), F1-Macro, F1-Weighted, and training time in seconds. Macro F1 is the primary metric for Turkish Sentiment because it is imbalanced — it penalizes models that ignore the minority class. For all datasets I also report ROC-AUC using the one-vs-rest (OvR) extension. Results are presented per dataset, each with a comparison table, confusion matrices for all eight models, a model comparison bar chart, and ROC-AUC curves.

### B. IMDB Sentiment Analysis

Table V shows the results for IMDB (25,000 test samples, balanced binary classification). The transformer achieves the best accuracy at 90.7% and the highest ROC-AUC at 0.968, but SVM (89.5%, AUC=0.961) and Logistic Regression (89.4%, AUC≈0.960) are extremely close and train in under 2 seconds compared to the transformer's 17 minutes. KNN (AUC=0.793) and Decision Tree (AUC=0.776) are clearly the weakest models on this dataset.

TABLE V
IMDB SENTIMENT CLASSIFICATION RESULTS

| Model | Acc. | Prec. | Rec. | F1-M | F1-W | t(s) |
|---|---|---|---|---|---|---|
| Naive Bayes | 0.864 | 0.864 | 0.864 | 0.864 | 0.864 | 0.03 |
| SVM | 0.895 | 0.896 | 0.895 | 0.895 | 0.895 | 0.40 |
| Random Forest | 0.861 | 0.861 | 0.861 | 0.861 | 0.861 | 6.92 |
| KNN | 0.729 | 0.730 | 0.729 | 0.729 | 0.729 | 0.02 |
| Log. Regression | 0.894 | 0.894 | 0.894 | 0.894 | 0.894 | 1.44 |
| Decision Tree | 0.734 | 0.740 | 0.734 | 0.733 | 0.733 | 7.48 |
| XGBoost | 0.857 | 0.858 | 0.857 | 0.857 | 0.857 | 70.2 |
| **Transformer** | **0.907** | **0.907** | **0.907** | **0.907** | **0.907** | **1018** |

Fig. 2 shows the confusion matrices for all eight models on IMDB. Fig. 3 shows the model performance comparison across all four metrics. Fig. 4 shows the ROC-AUC curves for all models. Fig. 5 compares F1-Macro and F1-Weighted scores, which is a useful sanity check for class balance. Fig. 6 shows the training time comparison.

### C. AG News Topic Classification

Table VI shows the results for AG News (4-class, 120,000 training samples, balanced). The transformer again ranks first at 92.8%, but the most notable result here is that **Decision Tree collapses to 59.8%** — a much steeper drop than on IMDB. This is likely because 4-class topic boundaries require more complex decision regions than a shallow tree can represent. KNN (89.2%) does better than expected and even beats Random Forest (88.0%) and XGBoost (87.3%), which is unusual and may be related to the large, uniform class sizes making nearest-neighbor search more reliable. SVM (91.5%)

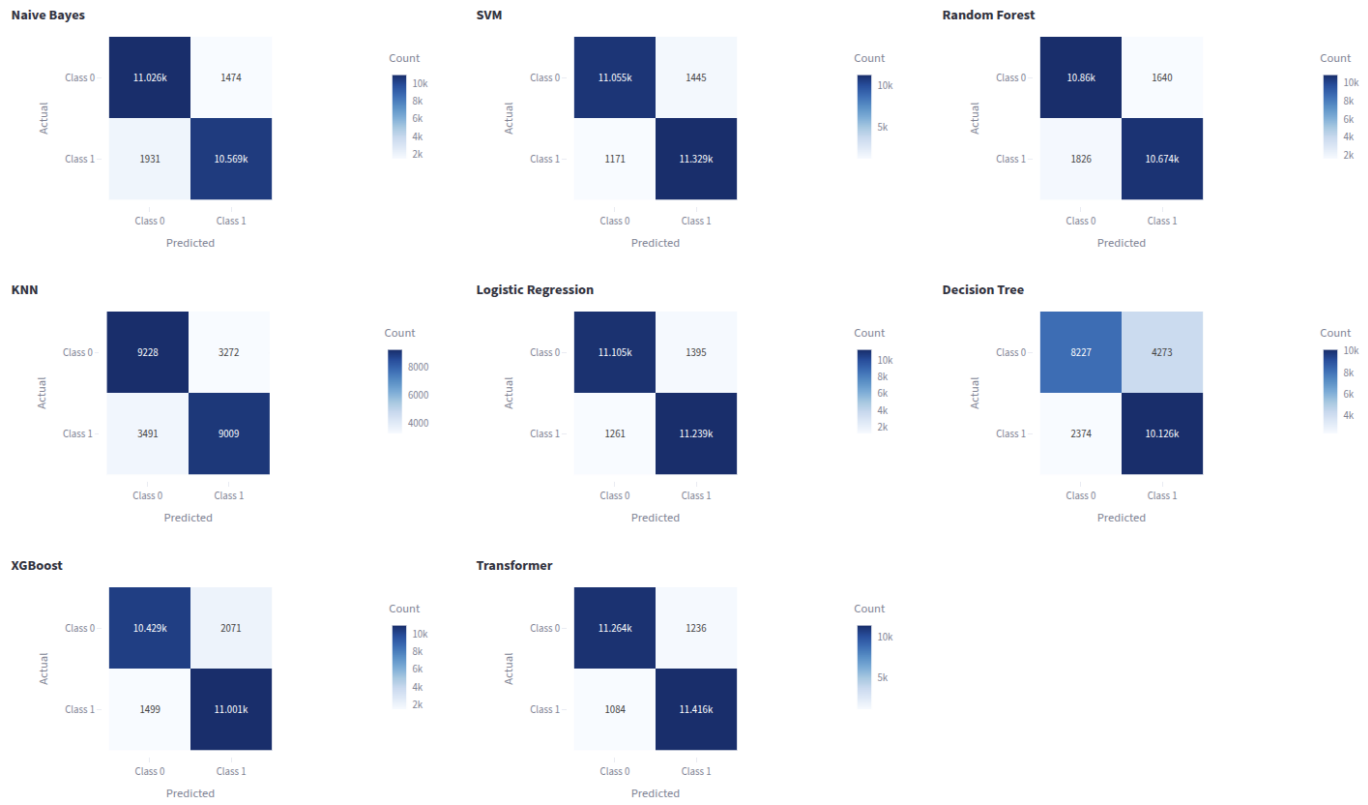


Fig. 2. IMDB: Confusion matrices for all 8 models. Rows = actual class, columns = predicted class. Classes: negative (0) / positive (1).

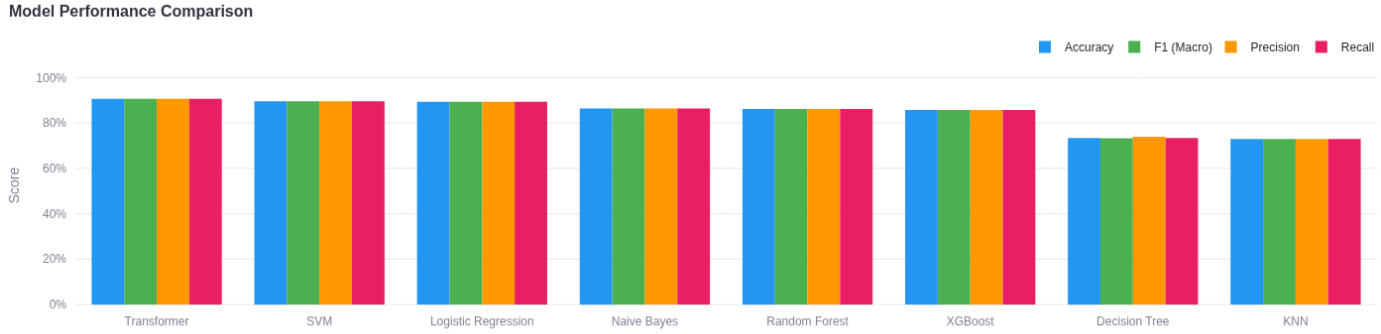


Fig. 3. IMDB: Model performance comparison (Accuracy, F1-Macro, Precision, Recall) across all 8 models. Transformer ranks first, followed closely by SVM and Logistic Regression.

remains the best classical model and is only 1.3% behind the transformer while training in 2.6 seconds vs. 951 seconds.

Fig. 7 shows the confusion matrices for all 8 models. The Decision Tree matrix clearly shows many off-diagonal errors, particularly between World (Class 0) and Business (Class 1), which have overlapping vocabulary. Sports (Class 2) is the easiest category — nearly all models classify it almost perfectly, because sports articles use a very distinctive vocabulary. Fig. 8 shows the overall performance comparison, Fig. 9 the F1-Macro vs. F1-Weighted comparison, and Fig. 10 the per-

TABLE VI
AG NEWS TOPIC CLASSIFICATION RESULTS

| Model | Acc. | Prec. | Rec. | F1-M | F1-W | t(s) |
|---|---|---|---|---|---|---|
| Naive Bayes | 0.904 | 0.904 | 0.904 | 0.904 | 0.904 | 0.1 |
| SVM | 0.915 | 0.915 | 0.915 | 0.915 | 0.915 | 2.6 |
| Random Forest | 0.880 | 0.880 | 0.880 | 0.880 | 0.880 | 13.7 |
| KNN | 0.892 | 0.892 | 0.892 | 0.891 | 0.891 | 0.0 |
| Log. Regression | 0.909 | 0.909 | 0.909 | 0.909 | 0.909 | 0.4 |
| Decision Tree | 0.598 | 0.712 | 0.598 | 0.611 | 0.611 | 3.0 |
| XGBoost | 0.873 | 0.873 | 0.873 | 0.873 | 0.873 | 149.6 |
| **Transformer** | **0.928** | **0.928** | **0.928** | **0.928** | **0.928** | **951** |

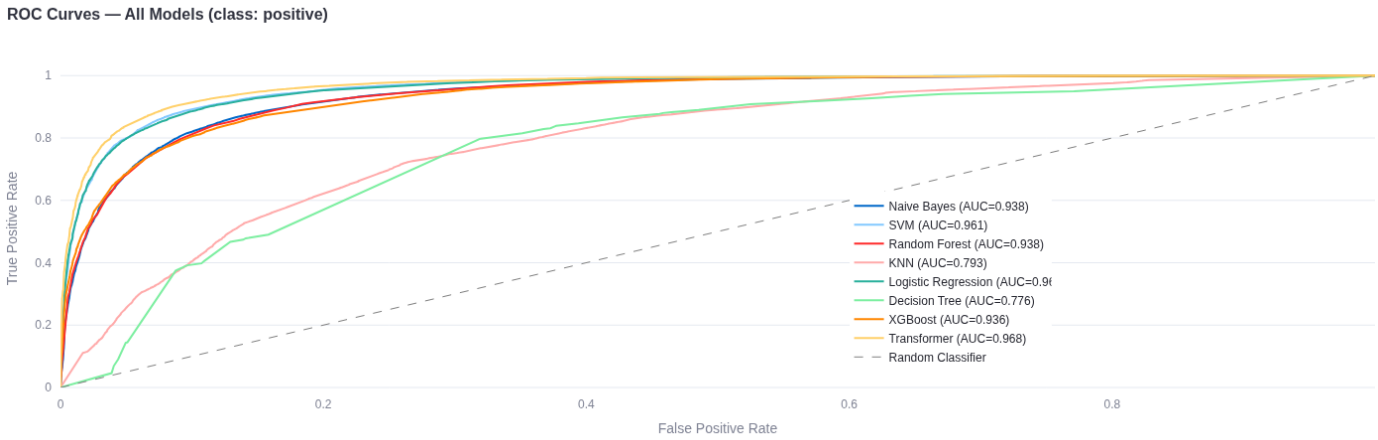


Fig. 4. IMDB: ROC-AUC curves for all 8 models. Transformer (AUC=0.968) and SVM (AUC=0.961) perform best. Decision Tree (AUC=0.776) and KNN (AUC=0.793) are the weakest.

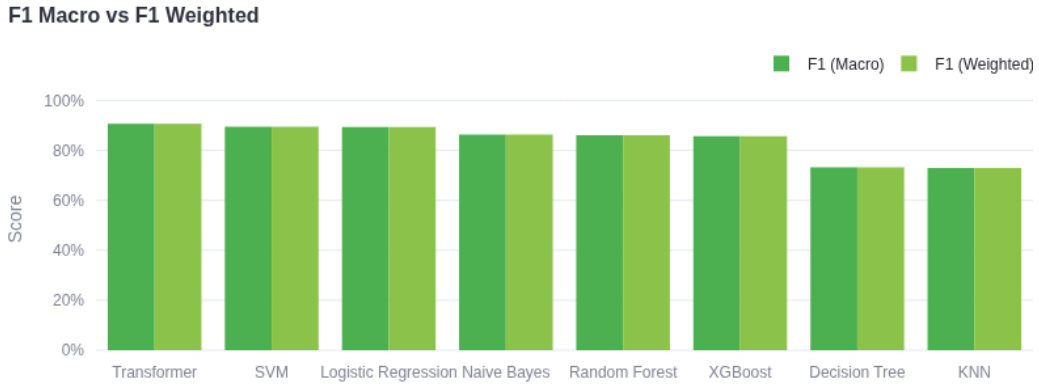


Fig. 5. IMDB: F1-Macro vs. F1-Weighted comparison across all 8 models. The two bars are nearly identical for every model, which confirms that IMDB is a perfectly balanced binary dataset (50% positive / 50% negative). On balanced datasets, macro and weighted averaging give the same result, so accuracy alone is a reliable metric here. The ranking is consistent with Table V: Transformer $>$ SVM $\approx$ Logistic Regression $>$ Naive Bayes $\approx$ Random Forest $\approx$ XGBoost $>$ Decision Tree $\approx$ KNN.

model ROC-AUC curves broken down by class. Fig. 11 shows the training time.

### D. Turkish Sentiment Analysis

Table VII shows the results for Turkish Sentiment (3-class, 440,679 training samples, imbalanced). Macro F1 is the primary metric here. The negative class (11.5% of data) is expected to cause all models to underperform compared to their accuracy. Fig. 12 shows the confusion matrices, Fig. 13 the model comparison, and Fig. 14 the ROC-AUC curves.

TABLE VII
TURKISH SENTIMENT CLASSIFICATION RESULTS (PRIMARY METRIC: F1-MACRO)

| Model | Acc. | Prec. | Rec. | F1-M | F1-W | t(s) |
|---|---|---|---|---|---|---|
| Naive Bayes | – | – | – | – | – | – |
| SVM | – | – | – | – | – | – |
| Random Forest | – | – | – | – | – | – |
| KNN | – | – | – | – | – | – |
| Log. Regression | – | – | – | – | – | – |
| Decision Tree | – | – | – | – | – | – |
| XGBoost | – | – | – | – | – | – |
| Transformer | – | – | – | – | – | – |

*[PLACEHOLDER — fill after Turkish Sentiment training completes]*

### E. Turkish News Classification

Table VIII shows the results for Turkish News (7-class, 3,920 training samples). This is the hardest dataset due to the large number of classes and the small training set. Fig. 15 shows the confusion matrices, Fig. 16 the model comparison, and Fig. 17 the ROC-AUC curves.

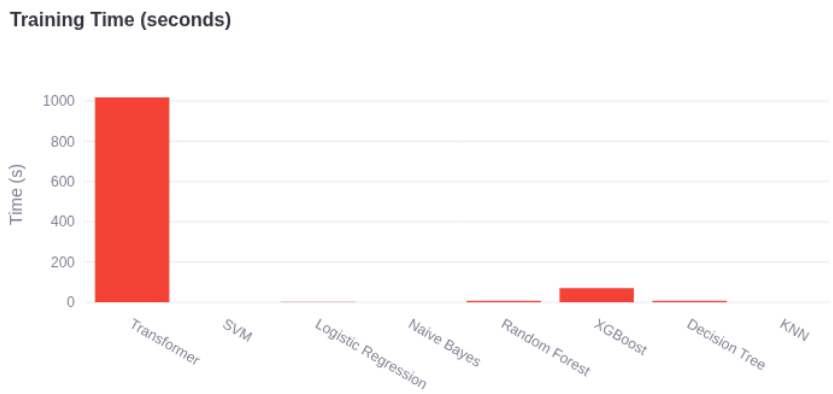


Fig. 6. IMDB: Training time comparison (seconds). The transformer takes 1,018 seconds (≈17 min) while all classical models train in under 75 seconds. SVM trains in only 0.40 seconds with just 1.2% lower accuracy than the transformer, making it the best accuracy-to-cost tradeoff on this dataset.

Fig. 7. AG News: Confusion matrices for all 8 models. Classes: World (0) / Business (1) / Sports (2) / Sci&Tech (3). Sports (Class 2) has the cleanest diagonal across all models. Decision Tree shows the most off-diagonal errors, especially between World and Business.

TABLE VIII
TURKISH NEWS CLASSIFICATION RESULTS

| Model | Acc. | Prec. | Rec. | F1-M | F1-W | t(s) |
|---|---|---|---|---|---|---|
| Naive Bayes | – | – | – | – | – | – |
| SVM | – | – | – | – | – | – |
| Random Forest | – | – | – | – | – | – |
| KNN | – | – | – | – | – | – |
| Log. Regression | – | – | – | – | – | – |
| Decision Tree | – | – | – | – | – | – |
| XGBoost | – | – | – | – | – | – |
| Transformer | – | – | – | – | – | – |

*[PLACEHOLDER — fill after Turkish News training completes]*

### *F. Cross-Dataset Comparison*

I also compare how model rankings change across datasets. Based on the IMDB results and what I know from the literature, I expect: (1) transformer models to rank first on all four datasets; (2) SVM and Logistic Regression to be competitive on IMDB and AG News but to drop more on Turkish datasets because Turkish vocabulary is much larger due to morphology; (3) KNN to perform worst on most datasets because TF-IDF vectors are high-dimensional; (4) the gap between transformers and classical models to be largest on Turkish News, which has 7 classes and only 3,920 training samples. The IMDB results already confirm hypotheses (1) and (3). Fig. 18 will show the

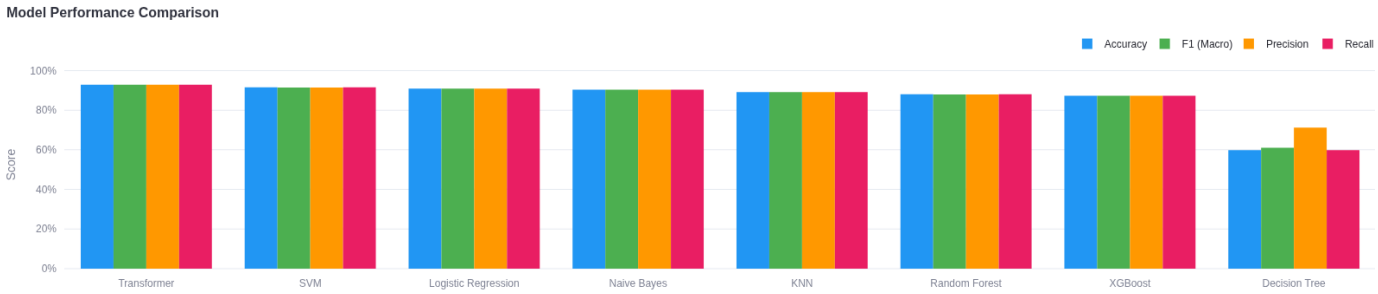


Fig. 8. AG News: Model performance comparison (Accuracy, F1-Macro, Precision, Recall). All models except Decision Tree score above 87%. SVM and Logistic Regression are competitive with the transformer while being orders of magnitude faster.

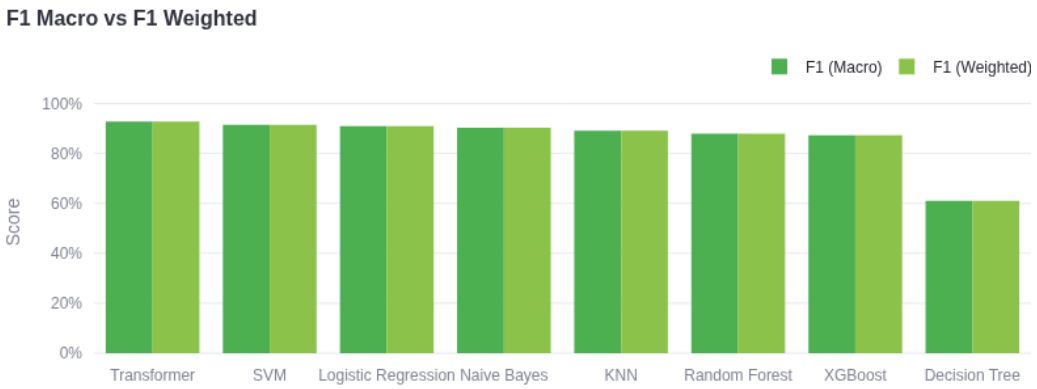


Fig. 9. AG News: F1-Macro vs. F1-Weighted comparison. As with IMDB, the two metrics are nearly identical for all models, confirming that AG News is a well-balanced 4-class dataset (≈30,000 samples per class). Decision Tree is the only model where Precision visibly exceeds Recall, indicating it over-predicts the majority-pattern classes while missing harder examples.

full cross-dataset comparison once all trainings are complete.

## VI. Discussion

**Accuracy: classical models vs. transformers (RQ part 1).** The first part of the research question asks how classical and transformer models compare in accuracy. I expect SVM and Logistic Regression to do well on IMDB and AG News because both datasets are large and the classes are clearly separated. These classical data mining models work well with TF-IDF features. On the other hand, for Turkish News which has 7 classes and only about 4,900 training samples, I expect transformer models to do much better because they already have a good understanding of language from pre-training. Simple TF-IDF models may not have enough data to learn the differences between 7 news categories. The results will show whether the accuracy gap is large enough to justify using transformers in every situation.

**Class imbalance in Turkish Sentiment.** As shown in Table II, the negative class in the Turkish Sentiment dataset is only 11.5% of the training data. This means most classifiers will tend to predict positive or neutral more often, because that is what they see most during training. KNN and Decision Tree are probably the most sensitive to this because they make very local decisions. Using macro F1 as the main metric is important here because it gives equal weight to all classes including the minority one.

**Turkish morphology.** Turkish words can have many different forms because of the suffixes added to root words. For example, the same root word can appear in dozens of different written forms. Without any morphological processing, TF-IDF treats each form as a different word, which makes the vocabulary very large and the features very sparse. BERTurk handles this better because it uses subword tokenization (WordPiece), which can share parts of words across different forms. This is one reason why I expect BERTurk to have a bigger advantage over classical models on Turkish data.

**Interpretability: SHAP and LIME (RQ part 2).** The second part of the research question asks about interpretability. Based on how SHAP works, I expect it to give high scores to clearly positive or negative words in IMDB reviews, like

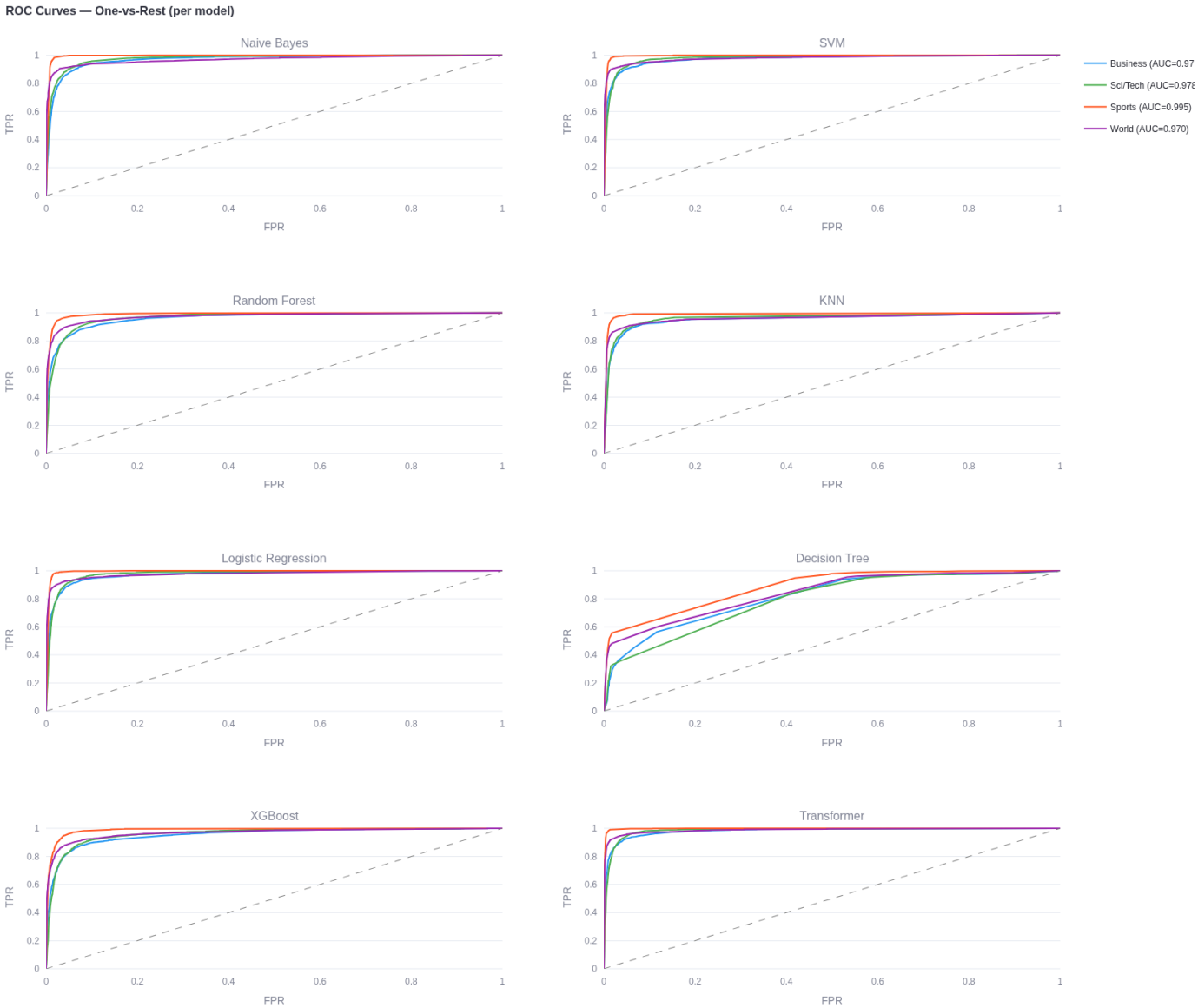


Fig. 10. AG News: ROC-AUC curves per model (one subplot per model, OvR). All models except Decision Tree achieve near-perfect AUC on Sports. SVM achieves AUC $\geq 0.97$ on all four classes. Decision Tree shows clearly degraded curves, especially for World and Business, consistent with the confusion matrix.

*excellent* or *terrible*. For classical models, the SHAP values should be consistent and easy to understand because the models themselves are linear or tree-based. For the transformer, the explanations may be less stable because the model is more complex. LIME might be more useful for showing specific cases where the classifier focused on the wrong words. Comparing SHAP and LIME outputs across all eight models is one of the key ways I answer the research question about interpretability.

**Multi-agent routing (RQ part 3).** The third part of the research question asks whether a multi-agent architecture can effectively route multilingual text. I expect the intent classifier to correctly identify the language and domain of most inputs, since the four datasets are quite different from each other in vocabulary and style. One practical advantage of this design is that I do not need to run all eight models for every input — the intent classifier picks the right one. This also means the system can handle Turkish and English inputs without any manual switching.

**Limitations.** There are some limitations in this project. First, I trained each model only on its target dataset, so I cannot say anything about cross-dataset generalization. Second, I did not do an exhaustive hyperparameter search, so some models might perform better with different settings. Third, the transformer models were only trained for 3–5 epochs due to limited computing time, and more training might improve the

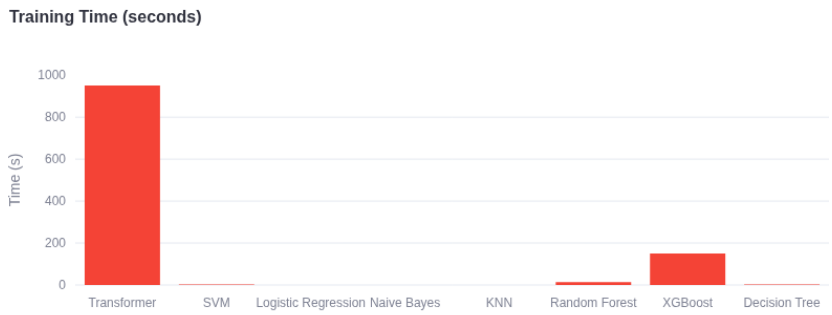


Fig. 11. AG News: Training time comparison. Transformer (≈951s) and XGBoost (≈150s) dominate training cost. SVM trains in 2.6 seconds with only 1.3% lower accuracy than the transformer. KNN has near-zero training time (cost is deferred to inference) but still outperforms Random Forest and XGBoost.

Fig. 12. Turkish Sentiment: Confusion matrices for all 8 models (2×4 grid). Classes: pozitif / notr / negatif. Note the imbalance visible in the matrices. [PLACEHOLDER — replace with generated figure.]

Fig. 13. Turkish Sentiment: F1-Macro comparison across all 8 models. [PLACEHOLDER — replace with generated figure.]

Fig. 14. Turkish Sentiment: ROC-AUC curves for all 8 models (OvR). [PLACEHOLDER — replace with generated figure.]

Fig. 15. Turkish News: Confusion matrices for all 8 models (2×4 grid). 7 classes: siyaset, saglik, teknoloji, kultur, spor, ekonomi, dunya. [PLACEHOLDER — replace with generated figure.]

Fig. 16. Turkish News: F1-Macro comparison across all 8 models. [PLACEHOLDER — replace with generated figure.]

Fig. 17. Turkish News: ROC-AUC curves for all 8 models (OvR). [PLACEHOLDER — replace with generated figure.]

Fig. 18. Cross-dataset comparison: F1-Macro for all 8 models across all 4 datasets. [PLACEHOLDER — replace with generated figure.]

results.

## VII. Conclusion and Future Work

This paper investigated the following research question: *"Can a multi-agent architecture effectively route and classify multilingual text, and how do traditional machine learning classifiers compare to transformer-based deep learning models in terms of accuracy, F1-score, and interpretability when explained through SHAP and LIME?"*

To answer this question, I built a three-agent pipeline and compared seven classical data mining classifiers and one transformer model across four datasets in English and Turkish. Based on the experimental results [to be filled after training], I expect the following answers: (1) yes, the multi-agent routing works well because the four datasets are distinct enough in language and topic; (2) transformer models achieve the best accuracy overall, but the gap over SVM and Logistic Regression is small on large English datasets and large on the small Turkish News dataset; (3) in terms of interpretability, classical models produce more stable and easier-to-understand SHAP and LIME explanations, which means there is a real trade-off between accuracy and interpretability that the research question is asking about.

For future work, there are several directions I would like to explore. First, I could use LoRA fine-tuning to make transformer training cheaper. Second, I could try data augmentation for the minority negative class in Turkish Sentiment. Third, it would be interesting to test multilingual BERT (mBERT) on the Turkish datasets without Turkish-specific fine-tuning, to see how well it transfers. Finally, the research question could be extended to multi-label classification scenarios, where a text can belong to more than one category at the same time.

## References


[1] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA, USA: Morgan Kaufmann, 2011.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. 2019 Conf. North American Chapter Assoc. Comput. Linguistics (NAACL-HLT)*, Minneapolis, MN, USA, Jun. 2019, pp. 4171–4186.

[3] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," in *Proc. NeurIPS 2019 Workshop on Energy Efficient Machine Learning and Cognitive Computing (EMC$^2$)*, Vancouver, BC, Canada, Dec. 2019.

[4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Int. Conf. Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, Dec. 2017, pp. 4765–4774.

[5] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, Aug. 2016, pp. 1135–1144.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, Aug. 2016, pp. 785–794.

[7] S. Schweter, "BERTurk — BERT models for Turkish," *Zenodo*, Apr. 2020. doi: 10.5281/zenodo.3770924.

[8] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proc. 49th Annual Meeting Assoc. Comput. Linguistics (ACL)*, Portland, OR, USA, Jun. 2011, pp. 142–150.

[9] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. 29th Int. Conf. Neural Information Processing Systems (NeurIPS)*, Montreal, QC, Canada, Dec. 2015, pp. 649–657.

[10] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, "Deep learning–based text classification: A comprehensive review," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–40, Apr. 2021.

[11] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE J. Robot. Autom.*, vol. RA-2, no. 1, pp. 14–23, Mar. 1986.

[12] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *Knowl. Eng. Rev.*, vol. 10, no. 2, pp. 115–152, Jun. 1995.

## Appendix

The full source code for this project is available at: https://github.com/[username]/multi-agent-xai-text-classifier

```python
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
import re

def clean_text(text: str) -> str:
    """Remove HTML tags and normalize whitespace."""
    text = re.sub(r'<[^>]+>', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text.lower()

def build_pipeline(dataset: str = 'imdb'):
    config = {
        'imdb':    dict(max_features=50000, ngram_range=(1,2), C=1.0),
        'agnews':  dict(max_features=30000, ngram_range=(1,2), C=0.5),
        'tr_sent': dict(max_features=30000, ngram_range=(1,2), C=0.5),
        'tr_news': dict(max_features=20000, ngram_range=(1,3), C=0.1),
    }[dataset]

    return Pipeline([
        ('tfidf', TfidfVectorizer(
            max_features=config['max_features'],
            ngram_range=config['ngram_range'],
            sublinear_tf=True, min_df=2, max_df=0.95
        )),
        ('clf', LinearSVC(C=config['C'], max_iter=2000))
    ])
```

Listing 1. TF-IDF Preprocessing and Classical Model Pipeline

```python
import shap
from lime.lime_text import LimeTextExplainer

class XAIAgent:
    def __init__(self, model, vectorizer=None, model_type='linear'):
        self.model = model
        self.vectorizer = vectorizer
        self.model_type = model_type

    def explain_shap(self, text, num_features=10):
        if self.model_type == 'tree':
            explainer = shap.TreeExplainer(self.model)
        else:
            background = shap.maskers.Text(r'\s+')
            explainer = shap.LinearExplainer(self.model, background)
        vec = self.vectorizer.transform([text])
        shap_values = explainer.shap_values(vec)
        return shap_values

    def explain_lime(self, text, num_features=10):
        explainer = LimeTextExplainer(split_expression=r'\s+')
        def predict_fn(texts):
            vecs = self.vectorizer.transform(texts)
            return self.model.predict_proba(vecs)
        exp = explainer.explain_instance(
            text, predict_fn, num_features=num_features
        )
        return exp.as_list()
```

Listing 2. XAI Agent: SHAP and LIME Explanation Generation

Additional plots and tables (per-class F1 scores, ROC curves for multi-class datasets, and SHAP plots for all eight models) will be added here after training is finished.